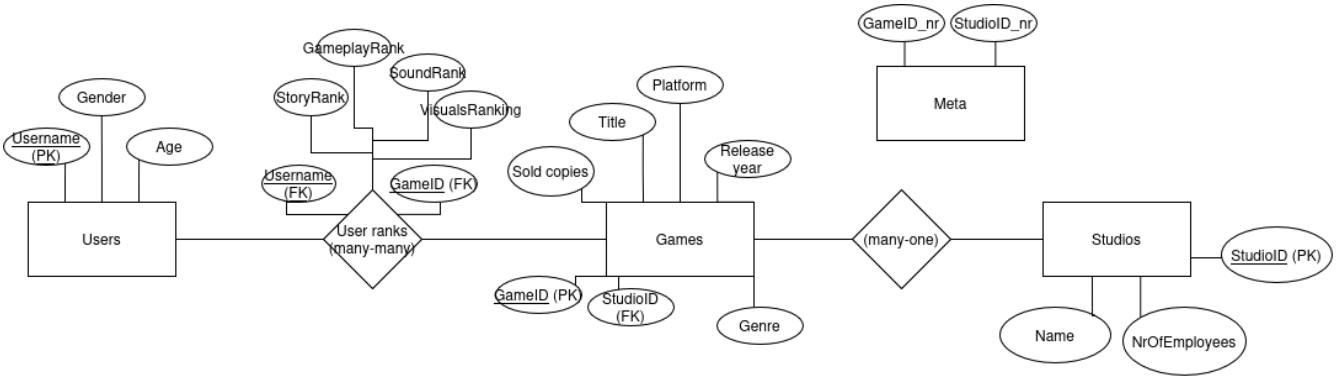
Game rating database

Name: Gabriel Abrahamsson Wiklund
Link to github: https://github.com/Exeboss02/GameRatingDataBase.git

1. Idea

I've made a database in mysql and python where users can rate their favorite games. It's made for those who play games and want to give them a rating so that other users can compare games they have played. The good thing about this database is that users cannot only see their own game ratings, they can also see any users ratings as well which makes it much more interesting. For this to be possible, there are some features the database must have to be able to provide this functionality. The most fundamental one is that you need to be able to insert a user-rating tied to a user and a game and what score you give it in gameplay, visuals, sound and story. But to be able to do that, you also offcourse have to be able to insert users with usernames, games with titles tied to a studio, and studios themselves who actually made the game. Users can only insert ratings for games that already exist in the database, and games can only be added if the studio who made it already exists in the database.

2. E/R diagram



In the E/R diagram above we can see three tables consisting of Users, Games and Studios, where all have different attributes and relations. The table Users have the attributes username, gender and age, where username is the primary key. A primary key is a unique identifier value that is tied to a single certain entry in a table. Then there is the Games table with 8 attributes. The primary key of Games is maybe surprisingly not the title attribute, but instead a GameID and the reason for this is to

allow several games with the same title to be present in the table, which is a use case in the real world. Each Games entry also has a studio-id, which is a foreign key to the Studios table so that each game can be tied to a single studio that created the game.

User ratings is what connects Users and Games, and is a relational table. The primary key of UserRatings consists of the two separate attributes username and game-id, which makes each rating tied to exactly one user and a single game. The relation between Games and Studios is much simpler where (as mentioned before) each game has studio_id as a foreign key that connects it to a single studio.

When looking at the mysql backend there also is another table not mentioned before that is used for keeping track of the next available id for Games and Studios entries. This table is called Meta and stores a single entry with the two values current-game-index and current-studio-index both starting at 0. There are two triggers tied to Meta, Games and Studios, where the current-game-index gets increased by 1 each time a new entry gets placed in Games, and the same with entries in Studios but instead with current-studio-index. The Meta table has no connections to other tables since there are no consistent values stored in it, it's just used internally to keep track of the number parts of the next id's. The game-id and studio-id are not single integer values though, instead game-id's are prefixed with a G then follows the id number making for example G5. Studio-id's work in the same way but is prefixed with an S instead.'

3. Schema for creating the database and tables

```
#----------SETUP-------------------------------------------------------
---
CREATE DATABASE GameRatings;
USE GameRatings;

CREATE TABLE Meta(
   CurrentGameIndex INT,
   CurrentStudioIndex INT
);

INSERT INTO Meta(CurrentGameIndex, CurrentStudioIndex) VALUES(0, 0);

#----------TABLES------------------------------------------------------
----

CREATE TABLE Users(
```

```
    UserName VARCHAR(255) PRIMARY KEY,
    Gender VARCHAR(16),
    Age INT,
    Country VARCHAR(255)
);

CREATE TABLE Studios(
    StudioID VARCHAR(32) PRIMARY KEY,
    Name VARCHAR(255),
    NrOfEmployees INT,
    Country VARCHAR(255)
);

CREATE TABLE Games(
    GameID VARCHAR(32) PRIMARY KEY,
    StudioID VARCHAR(32),
    FOREIGN KEY (StudioID) REFERENCES Studios(StudioID),
    Title VARCHAR(255),
    Platform VARCHAR(255),
    ReleaseYear INT,
    SoldCopies INT,
    Genre VARCHAR(32)
);

CREATE TABLE UserRatings(
    UserName VARCHAR(255),
    GameID VARCHAR(32),
    PRIMARY KEY (UserName, GameID),
    FOREIGN KEY (UserName) REFERENCES Users(UserName),
    FOREIGN KEY (GameID) REFERENCES Games(GameID),
    StoryRating INT,
    GamePlayRating INT,
    VisualsRating INT,
    SoundRating INT
);
```

There are procedures for inserting into Users, UserRatings, Games and Studios where insertion into UserRatings first checks that there actually is a user with specified username and a Games entry with specified game-id. Games insertion first checks that there is a studio with specified studio-id before the actual insertion, and if studio-id is not in the database then the procedure finishes without insertion. To

check if game-id, studio-id or username exists, there are the functions CheckGame, CheckStudio and CheckUser that will return 1 if the id exists and 0 if it doesn't. CheckUser is used for checking that a user with the specified username doesn't already exist before insertion so there are no duplicates or confusion.

4. Queries

Though there are many queries used in this database, five of them will be described below to give a better understanding of the functionality of this database. If the user wants to retrieve specific data from the database that isn't covered by the standard functions, there is custom query functionality to call any query they want from python.

Query 1:

```
DELIMITER //
CREATE PROCEDURE InsertUserRating(
   IN userName VARCHAR(255),
   IN gameID VARCHAR(32),
   IN storyRating INT,
   IN gamePlayRating INT,
   IN visualsRating INT,
   IN soundRating INT
)
BEGIN
   IF CheckUser(userName) = 0 OR CheckGame(gameID) = 0
      Then SELECT "The user or gameID doesn't exist!";
   ELSE
      INSERT INTO UserRatings(UserName, GameID, StoryRating, GamePlayRating,
VisualsRating, SoundRating)
         VALUES (userName, gameID, storyRating, gamePlayRating, visualsRating,
soundRating);
   END IF;
END //
DELIMITER ;
```

This query is for creating the InsertUserRating procedure. It takes several inputs like username and gameid to be able to link the rating to the correct user and game. It first checks so that both the user and the gameid exist in the database with two functions called CheckUser and CheckGame. The rating will be inserted in the UserRatings table if both the user and game exists.

Query 2:

```
SELECT Games.Title,
       AVG(StoryRating) AS avgStoryScore,
       AVG(GamePlayRating) AS avgGamePlayScore,
```

```
    AVG(VisualsRating) AS avgVisualsScore,
    AVG(SoundRating) AS avgSoundScore
FROM
    UserRatings
INNER JOIN
    Games ON Games.GameID = UserRatings.GameID
WHERE
    Games.GameID = '""" + game_id + """'
GROUP BY Games.Title;
```

This query is found in the Get_Avg_Ratings_For_Game python function. It takes the average score of each category from all user ratings for a certain game and lists them. It takes the title and average ratings from UserRatings and combines the UserRatings and Games tables with INNER JOIN where gameID is matching, which gives access to the game title that is stored in the Games table (1st join). game_id is a parameter to this function.

Query 3:

```
SELECT GameID, AVG(GamePlayRating) FROM UserRatings GROUP BY
UserRatings.GameID
```

This query is called in the python re-initialize function and uses a GROUP operation together with AVG to calculate the average GamePlayRating for each game in the database identified by GameID. The GROUP operation makes the results be per game.

Query 4:

```
SELECT Games.*, Studios.Name FROM Games INNER JOIN Studios ON
Games.StudioID = Studios.StudioID WHERE Studios.StudioID = '" +
studio_id + "';
```

This query is called from python in the Games_By_Studio function, and it lists all games that a studio with specified studio_id has made. It uses an INNER JOIN operation to combine the Games and Studios tables based on matching StudioID's, and only selects the ones where StudioID matches the specified studio_id (2nd join). studio_id is a parameter to this function.

Query 5:

```
DELIMITER //
CREATE TRIGGER AfterAddingGame
AFTER INSERT ON Games
FOR EACH ROW
BEGIN
   UPDATE Meta SET Meta.CurrentGameIndex = Meta.CurrentGameIndex + 1;
END //
```

DELIMITER ;

This query is in the MYSQL setup and creates one of the triggers in the database. The trigger goes off each time something is inserted into the Games table, and it updates the CurrentGameIndex in the Meta table so each new Games entry gets a valid GameID. It is basically a counter that goes up each time a game is added, but is very useful since it automatically keeps track of what the next gameid should be.

5. Discussion and details

To use the database in a more accessible way, a python interface has been made. Functions for calling core functionality such as Insert_User and Insert_User_Rating etc. exist. You can also read and insert data for games and studios from .csv files to make the process easier with Insert_Games_From_CSV and Insert_Studios_From_CSV. These functions use the pandas library to read tables in csv-files. To connect to and call things to the database, mysql.connector library was used.

There is unfortunately no graphical user interface, so the user has to run the python file and then enter commands in the terminal. To create the database from scratch, you run all the queries in the .sql file with mysql workbench to create the database and its resources. Then you can run the “Re_Initialize” function in python to insert some basic entries in the tables. If you want to view certain data, you can either use pre-made functions, or you can call a custom query with “Call_Query” and provide a mysql query as a string. You can see all pre-made functions by typing “commands”.

The Call_Query python function example

```
enter command: call-query
----------------------------CALL-QUERY-------------------------------------------
    enter query(with correct mysql syntax): SELECT * FROM Studios
('S0', 'FromSoftware', 1000, 'Japan')
('S1', 'Bungie', 1000, 'USA')
('S2', 'Infinite Fall', 5, 'USA')
('S3', 'Toby Fox', 1, 'USA')
('S4', 'Crytek', 500, 'Germany')
('S5', 'Valve', 400, 'USA')
('S6', 'Psyonix', 200, 'USA')
('S7', 'Nintendo', 5000, 'Japan')
('S8', 'Team Cherry', 3, 'Australia')
('S9', 'Finji', 10, 'USA')
('S10', 'FromSoftware', 1000, 'Japan')
('S11', 'Bungie', 1000, 'USA')
```

I think the project went pretty well and I think most things work okay, but it would have made a big difference if there was a graphical user interface since it’s harder to make mistakes if you only have buttons to press instead of writing text based

commands or entering queries manually. MYSQL is fine to work with but feels old, so I'm glad there are more modern options available for managing databases. The integration with python has worked really well though.

Link to github: https://github.com/Exeboss02/GameRatingDataBase.git
Python libraries: pandas, mysql connector, numpy
Other: mysql workbench